

PROGRAMACIÓN LÓGICA Y FUNCIONAL:

SESIÓN 15

CONTENIDO

- Bienvenida
- Listas y cadenas
- Laboratorio

CONCEPTOS PREVIOS

¿Qué condición debe cumplirse para que una función pueda ser recursiva? Enuncie un ejemplo donde sea útil su uso

Levantemos la mano para participar 

LOGRO

Al finalizar la presente sesión el estudiante:

- Será capaz de manipular colecciones y texto en Haskell mediante recursividad y funciones de orden superior.



UTILIDAD

¿Porque es importante conocer las listas y el trabajo con cadenas en Haskell?



¿Qué son las listas?

En Haskell, una lista es una estructura homogénea (todos los elementos del mismo tipo) y recursiva.

- Sintaxis: `[1,2,3]` es en realidad la suma sintáctica de `1 : (2 : (3 : []))`.
- Operador Cons (`:`): Agrega un elemento al inicio de una lista.
- Cadenas (String): Son alias de `[Char]`. Por ende, `"Hola"` es igual a `['H','o','l','a']`.

¿Qué son las listas?

- Si es un tipo cualquiera, entonces representa el tipo de lista cuyos elementos son valores de tipo
- Se pueden escribir usando corchetes [] con comas los valores individuales de la lista. Así la expresión representa una lista de tres valores.
- También se permite expresar una lista de valores sucesivos, como el cual contiene los 11 números enteros del 10 al 20.
- Una lista vacía se expresa con los corchetes vacíos

Acceso y Descomposición

Para entender cómo viajar por una lista, usamos funciones básicas:

- head: Primer elemento.
- tail: La lista sin el primero.

Acceso y Descomposición

Crear una función que sume los dos primeros elementos de una lista.

```
sumarDosPrimeros :: [Int] -> Int
sumarDosPrimeros (x:y:_) = x + y
sumarDosPrimeros _      = 0
```

```
main :: IO ()
main = do
    --let numeros = [3, 5, 7, 9]
    let numeros = []
    let resultado = sumarDosPrimeros numeros
    putStrLn $ "La suma de los dos primeros
números es: " ++ show resultado
```

Nuestra propia función longitud

Crear una función que sume los dos primeros elementos de una lista.

```
miLongitud :: [a] -> Int
miLongitud []      = 0      -- Caso Base
miLongitud (_:xs) = 1 + miLongitud xs      --
Caso Recursivo: 1 más la longitud del resto
```

Operaciones fundamentales con listas

- **null** *[lista]*
 - Devuelve True si [lista] se encuentra vacía.
- **head** *[lista]*
 - Devuelve el primer valor de la lista.
- **tail** *[lista]*
 - Devuelve la lista de valores de lst después de su primer valor.

Funciones para Listas

- `length [lista]`
 - Devuelve el número de valores de `[lista]`
- `last [lista]`
 - Devuelve el valor final de la `[lista]`
- `init [lista]`
 - Devuelve una lista, menos el valor final de la `[lista]`
- `[lista] !! K`
 - Devuelve el valor del índice `K` de la `[lista]` (índice desde cero)
- `[Lista_1] ++ [Lista_2]`
 - Devuelve la concatenación de las listas

Funciones de Orden Superior

Son funciones que reciben otras funciones como argumento.

- map: Aplica una función a cada elemento.
- filter: Filtra según un predicado.

Funciones de Orden Superior

Dada una lista de palabras, obtener la longitud de aquellas que tengan más de 3 letras.

```
palabrasConLongitud :: [String] -> [(String, Int)]  
palabrasConLongitud lista =  
    map (\p -> (p, length p)) (filter (\p -> length p > 3) lista)
```

Funciones de Orden Superior

Dada una lista de palabras, obtener la longitud de aquellas que tengan más de 3 letras.

```
main :: IO ()
main = do
    let palabras = ["hola", "mundo", "hi", "programacion", "fun"]
    let resultado = palabrasConLongitud palabras

    putStrLn "Análisis de palabras (longitud > 3):"
    print resultado
```

Funciones de Orden Superior

Dada una lista de palabras, obtener la longitud de aquellas que tengan más de 3 letras. Versión resumida

```
palabrasConLongitud' :: [String] -> [(String, Int)]
palabrasConLongitud' lista = [(p, length p) | p <- lista, length p > 3]
main :: IO ()
main = do
    let palabras = ["hola", "mundo", "hi", "programacion", "fun"]
    let resultado = palabrasConLongitud' palabras

    putStrLn "Análisis de palabras (longitud > 3):"
    print resultado
```


Cadenas (Strings) y el tipo Char

Haskell trata las cadenas como `type String = [Char]`. Esto permite polimorfismo total.

- Funciones de `Data.Char`: Es vital introducir funciones como `toUpper`, `isDigit`, `isAlpha`.
- Conversión: Uso de `show` (de cualquier tipo a `String`) y `read` (de `String` a otro tipo).

Validaciones

Validar si una cadena es un número de teléfono

```
import Data.Char (isDigit)
```

```
esTelefono :: String -> Bool
```

```
esTelefono [] = False
```

```
esTelefono s = all isDigit s -- 'all' es una función de alto nivel sobre listas
```

List Comprehensions: Filtrado y Transformación Avanzada

Más allá de lo básico, podemos usar múltiples generadores y guardas.

- **Producto Cartesiano:** Generar todas las combinaciones posibles.

List Comprehensions: Filtrado y Transformación Avanzada

Generar todas las coordenadas (x, y) de un tablero de ajedrez donde la suma sea par.

```
ajedrezPares :: [(Int, Int)]  
ajedrezPares = [(x, y) | x <- [1..8], y <- [1..8], even (x + y)]
```

```
main :: IO ()  
main = do  
    putStrLn "Pares de coordenadas en un tablero de ajedrez (x, y):"  
    print ajedrezPares
```

Ejercicios

Crear una multiplicación de lista

Recursión Manual (El "Cómo" funciona)

Es la base lógica. Útil para explicar el *Pattern Matching* y el caso base.

```
multiplicarR :: [Int] -> Int  
multiplicarR [] = 1           -- Caso Base (Identidad)  
multiplicarR (x:xs) = x * multiplicarR xs -- Caso Recursivo
```

Ejercicios

Crear una multiplicación de lista

Plegado por la Izquierda (foldl)

Representa la abstracción de la recursión. El acumulador viaja de izquierda a derecha.

```
multiplicarL :: [Int] -> Int  
multiplicarL lista = foldl (*) 1 lista
```

Ejercicios

Crear una multiplicación de lista

Plegado por la derecha (foldl)

Similar al anterior, pero procesa desde el final. En Haskell, es preferible para listas infinitas o perezosas.

```
multiplicarL :: [Int] -> Int  
multiplicarL lista = foldr (*) 1 lista
```

Ejercicios

Crear una multiplicación de lista

Función Nativa (product)

La forma más eficiente y legible en entornos profesionales.

```
multiplicarN :: [Int] -> Int  
multiplicarN = product
```


Actividad grupal

Dada una lista de palabras, filtrar las que tienen más de 3 caracteres, obtener sus longitudes y devolver el producto total de esas longitudes.

```
-- 1. Filtramos palabras largas y obtenemos sus longitudes
obtenerLongitudes :: [String] -> [Int]
obtenerLongitudes lista = [length p | p <- lista, length p > 3]

-- 2. Multiplicamos todos los elementos de la lista resultante
multiplicarElementos :: [Int] -> Int
multiplicarElementos = product
```

Actividad grupal

Dada una lista de palabras, filtrar las que tienen más de 3 caracteres, obtener sus longitudes y devolver el producto total de esas longitudes.

```
main :: IO ()
main = do
    let palabras = ["hola", "mundo", "hi", "programacion"]
    let longitudes = obtenerLongitudes palabras
    let resultadoFinal = multiplicarElementos longitudes

    putStrLn $ "Lista de palabras: " ++ show palabras
    putStrLn $ "Longitudes (>3): " ++ show longitudes
    putStrLn $ "El producto de las longitudes es: " ++ show resultadoFinal
```

Qué hemos aprendido el día de hoy?



CONCLUSIONES

- Las listas en Haskell permiten pensar en el "qué" (declarativo) y no en el "cómo" (procedimental).
- Podemos encadenar funciones (map, filter, fold) como piezas de Lego.
- El sistema de tipos de Haskell previene errores de "índice fuera de rango" en tiempo de compilación si se usa recursión correctamente.



REFERENCIAS

Bird, R. (2014). *Thinking Functionally with Haskell*. Cambridge University Press. **Capítulo 3: "Lists"**.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100
101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191	192	193	194	195	196	197	198	199	200
201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	281	282	283	284	285	286	287	288	289	290	291	292	293	294	295	296	297	298	299	300
301	302	303	304	305	306	307	308	309	310	311	312	313	314	315	316	317	318	319	320	321	322	323	324	325	326	327	328	329	330	331	332	333	334	335	336	337	338	339	340	341	342	343	344	345	346	347	348	349	350	351	352	353	354	355	356	357	358	359	360	361	362	363	364	365	366	367	368	369	370	371	372	373	374	375	376	377	378	379	380	381	382	383	384	385	386	387	388	389	390	391	392	393	394	395	396	397	398	399	400
401	402	403	404	405	406	407	408	409	410	411	412	413	414	415	416	417	418	419	420	421	422	423	424	425	426	427	428	429	430	431	432	433	434	435	436	437	438	439	440	441	442	443	444	445	446	447	448	449	450	451	452	453	454	455	456	457	458	459	460	461	462	463	464	465	466	467	468	469	470	471	472	473	474	475	476	477	478	479	480	481	482	483	484	485	486	487	488	489	490	491	492	493	494	495	496	497	498	499	500
501	502	503	504	505	506	507	508	509	510	511	512	513	514	515	516	517	518	519	520	521	522	523	524	525	526	527	528	529	530	531	532	533	534	535	536	537	538	539	540	541	542	543	544	545	546	547	548	549	550	551	552	553	554	555	556	557	558	559	560	561	562	563	564	565	566	567	568	569	570	571	572	573	574	575	576	577	578	579	580	581	582	583	584	585	586	587	588	589	590	591	592	593	594	595	596	597	598	599	600
601	602	603	604	605	606	607	608	609	610	611	612	613	614	615	616	617	618	619	620	621	622	623	624	625	626	627	628	629	630	631	632	633	634	635	636	637	638	639	640	641	642	643	644	645	646	647	648	649	650	651	652	653	654	655	656	657	658	659	660	661	662	663	664	665	666	667	668	669	670	671	672	673	674	675	676	677	678	679	680	681	682	683	684	685	686	687	688	689	690	691	692	693	694	695	696	697	698	699	700
701	702	703	704	705	706	707	708	709	710	711	712	713	714	715	716	717	718	719	720	721	722	723	724	725	726	727	728	729	730	731	732	733	734	735	736	737	738	739	740	741	742	743	744	745	746	747	748	749	750	751	752	753	754	755	756	757	758	759	760	761	762	763	764	765	766	767	768	769	770	771	772	773	774	775	776	777	778	779	780	781	782	783	784	785	786	787	788	789	790	791	792	793	794	795	796	797	798	799	800
801	802	803	804	805	806	807	808	809	810	811	812	813	814	815	816	817	818	819	820	821	822	823	824	825	826	827	828	829	830	831	832	833	834	835	836	837	838	839	840	841	842	843	844	845	846	847	848	849	850	851	852	853	854	855	856	857	858	859	860	861	862	863	864	865	866	867	868	869	870	871	872	873	874	875	876	877	878	879	880	881	882	883	884	885	886	887	888	889	890	891	892	893	894	895	896	897	898	899	900
901	902	903	904	905	906	907	908	909	910	911	912	913	914	915	916	917	918	919	920	921	922	923	924	925	926	927	928	929	930	931	932	933	934	935	936	937	938	939	940	941	942	943	944	945	946	947	948	949	950	951	952	953	954	955	956	957	958	959	960	961	962	963	964	965	966	967	968	969	970	971	972	973	974	975	976	977	978	979	980	981	982	983	984	985	986	987	988	989	990	991	992	993	994	995	996	997	998	999	1000



GRACIAS